

MARKET BASKET ANALYSIS

The Apriori Algorithm, Support, Confidence & Lift

Introduction to Market Basket Analysis

Market Basket Analysis (MBA) is a data mining technique used to discover relationships between products/items that customers purchase together.

It is widely used in:

- Retail
- E-commerce
- Recommendation systems
- Supermarkets
- Online shopping

Example

Suppose we have these transactions:

Transaction Items

T1	Bread, Milk
T2	Bread, Butter, Milk
T3	Bread, Butter
T4	Milk, Butter
T5	Bread, Milk, Butter

We may discover:

Customers who purchase **Bread and Milk** frequently also purchase **Butter**.

This relationship can be represented as:

Bread + Milk $\rightarrow$ Butter

This is called an **association rule**.

Association Rule Mining

Market Basket Analysis is an application of **Association Rule Mining**.

The goal is to discover rules of the form:

$X \rightarrow Y$

where:

- X = antecedent / left-hand side
- Y = consequent / right-hand side

For example:

$\{\text{Bread, Milk}\} \rightarrow \{\text{Butter}\}$

means:

When Bread and Milk are purchased, Butter is also frequently purchased.

The rule does **not** mean that buying Bread causes someone to buy Butter.

It only means that the two purchasing behaviours are associated according to the data.

This distinction is important.

Important Terminology

Transaction

A single customer's purchase is called a **transaction**.

Example:

$T1 = \{\text{Bread, Milk, Butter}\}$

Item

An individual product.

Examples:

Bread

Milk

Butter

Eggs

Cheese

Itemset

A collection of one or more items.

1-itemset

{Bread}

2-itemset

{Bread, Milk}

3-itemset

{Bread, Milk, Butter}

The number of items determines the size of the itemset.

Frequent Itemset

An itemset is called **frequent** if its support is greater than or equal to a predefined minimum support.

For example:

minimum support = 40%

If:

{Bread, Milk}

appears in 60% of transactions, it is a frequent itemset.

Core Metrics: Support, Confidence, and Lift

Every association rule $X \rightarrow Y$ is judged using three fundamental measures. These metrics decide which item sets are worth reporting and which discovered rules are actually meaningful rather than coincidental.

Support

Support measures how frequently an itemset appears in the entire transaction database.

It answers: "Out of all transactions, what fraction contain this itemset?"

$$\text{Support}(X) = (\text{Number of transactions containing } X) / (\text{Total number of transactions})$$

Example

Suppose we have 5 transactions:

Transaction Items

T1	Bread, Milk
T2	Bread, Butter, Milk
T3	Bread, Butter
T4	Milk, Butter
T5	Bread, Milk, Butter

Consider the itemset: {Bread, Milk}

It appears in:

T1

T2

T5

Therefore:

$$\begin{aligned}\text{Support}(B, \text{readMilk}) &= \frac{3}{5} \\ &= 0.60\end{aligned}$$

or: 60%

Support of a Rule

For a rule $X \rightarrow Y$, the rule's support is the support of the union of X and Y — i.e., the fraction of transactions containing both X and Y together:

$$\text{Support}(X \rightarrow Y) = \text{Support}(X \cup Y) = \text{count}(X \text{ and } Y \text{ together}) / |D|$$

Interpretation

High support means the itemset is popular / common across the whole dataset. A minimum support threshold is used to filter out itemsets that are too rare to be commercially or statistically interesting — it is a measure of significance, not of the strength of association.

Minimum Support

In Apriori, we normally define: `minimum_support = 0.4`

Then: `Support >= 0.4`

means the itemset is considered frequent.

Anything below the threshold is discarded.

Confidence

Confidence measures how often the rule has been found to be true — i.e., given that a transaction contains X , how likely is it to also contain Y ? It is a conditional probability.

It measures how often Y appears when X appears.

$$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X) = \text{count}(X \text{ and } Y) / \text{count}(X)$$

Example

Suppose:

Bread appears in 4 transactions

Bread + Butter appears in 3 transactions

Then:

$$\begin{aligned} \text{Confidence}(\text{Bread} \rightarrow \text{Butter}) &= \frac{3}{4} \\ &= 0.75 \end{aligned}$$

Therefore:

Confidence = 75%

Interpretation:

Among transactions containing Bread, 75% also contain Butter.

Interpretation

Confidence measures the strength/reliability of the implication, but it can be misleading on its own: a rule can have high confidence purely because Y is a very popular item overall (e.g., almost everyone buys Milk regardless of what else they buy). This is exactly the gap that Lift is designed to close.

Support vs Confidence

Suppose:

100 transactions

Bread appears in:

80 transactions

Bread + Butter appears in:

60 transactions

Then:

Support

$$\text{Support}(B, \text{readButter}) = \frac{60}{100} = 60\%$$

Confidence

$$\text{Confidence}(\text{Bread} \rightarrow \text{Butter}) = \frac{60}{80} = 75\%$$

So:

Support = 60%

Confidence = 75%

They are measuring different things.

Lift

Lift measures how much more often X and Y occur together than How often we would expect them to occur together if they were independent.

$$\begin{aligned} \text{Lift}(X \rightarrow Y) &= \text{Confidence}(X \rightarrow Y) / \text{Support}(Y) \\ &= \text{Support}(X \cup Y) / (\text{Support}(X) \times \text{Support}(Y)) \end{aligned}$$

Lift Value	Meaning	Practical Reading
Lift = 1	X and Y are independent	No real association; co-occurrence is by chance
Lift > 1	Positive correlation	X and Y are bought together more than expected — genuine cross-sell signal
Lift < 1	Negative correlation	Buying X makes Y less likely — items may be substitutes

Table 2. Interpreting Lift values.

Example of Lift

1. Suppose: Support (Bread) = 0.8 Support (Butter) = 0.5

Support (Bread, Butter) = 0.4

Confidence:

$$Confidence(Bread \rightarrow Butter) = \frac{0.4}{0.8} = 0.5$$

Therefore:

Confidence = 50%

Now calculate lift:

$$\begin{aligned} Lift &= \frac{0.5}{0.5} \\ &= 1 \end{aligned}$$

So Bread and Butter are effectively independent in this example.

2. Suppose: Support (Bread) = 0.5 Support (Butter) = 0.4 Support (Bread, Butter) = 0.3

Confidence:

$$Confidence = \frac{0.3}{0.5} = 0.6$$

Lift:

$$Lift = \frac{0.6}{0.4} = 1.5$$

Therefore: Lift = 1.5

Interpretation:

Customers who buy Bread are 1.5 times as likely to buy Butter compared with the baseline probability of buying Butter.

Quick Reference

Metric	Formula	Range	Question Answered
Support	$\text{count}(X \cup Y) / D $	0 to 1	How frequent/common is this pattern?
Confidence	$\text{count}(X \cup Y) / \text{count}(X)$	0 to 1	How reliable is the rule $X \rightarrow Y$?
Lift	$\text{Conf}(X \rightarrow Y) / \text{Supp}(Y)$	0 to ∞	How much better than random chance?

Table 3. Support, Confidence and Lift at a glance.

The Apriori Algorithm — Theory

Computing support for every possible itemset by brute force is computationally infeasible: with n distinct items there are $2^n - 1$ possible non-empty itemsets.

Apriori is an algorithm for finding **frequent itemsets** and generating association rules.

The Apriori (Anti-Monotone) Property

Key Principle

"All non-empty subsets of a frequent itemset must also be frequent." Equivalently, its contrapositive: if an itemset is infrequent, every superset of it is guaranteed to be infrequent too.

This property allows the algorithm to prune the search space aggressively: once it is known that {Diapers, Cola} does not meet the minimum support threshold, there is no need to ever check {Diapers, Cola, Beer}, {Diapers, Cola, Milk}, or any other superset — they cannot possibly be frequent either. This pruning is what makes Apriori practical on real datasets.

Algorithm Steps

1. Set a minimum support threshold (min_sup) and, for rule generation later, a minimum confidence threshold (min_conf).
2. Generate candidate 1-itemsets (C_1) — every individual item — and scan the database to compute the support of each.
3. Prune: keep only those items whose support $\geq \text{min_sup}$. This surviving set is the frequent 1-itemset list, L_1 .
4. Join step: generate candidate k -itemsets (C_k) by combining pairs of frequent $(k-1)$ -itemsets from L_{k-1} that share the first $k-2$ items.
5. Prune step: discard any candidate in C_k that contains an infrequent $(k-1)$ -subset (this is the anti-monotone check) — it cannot possibly be frequent.
6. Scan the database again to count actual support for the remaining candidates in C_k , and keep those meeting min_sup to form L_k .
7. Repeat steps 4–6, incrementing k , until no new frequent itemsets are found (L_k is empty).
8. The union of all L_k is the complete set of frequent itemsets, which is then used to generate association rules.

Pseudocode

```
L1 = {frequent 1-itemsets}
k = 2
while L(k-1) is not empty:
    Ck = candidates generated by joining L(k-1) with itself
    Ck = prune candidates whose (k-1)-subsets are not in L(k-1)
    for each transaction t in database D:
        increment count of every candidate in Ck
            that is a subset of t
    Lk = { c in Ck | support(c) >= min_sup }
    k = k + 1
Result = union of all Lk
```

Worked Example — Finding Frequent Itemsets

TID	Items Purchased
T1	Bread, Milk
T2	Bread, Diapers, Beer, Eggs
T3	Milk, Diapers, Beer, Cola
T4	Bread, Milk, Diapers, Beer
T5	Bread, Milk, Diapers, Cola
T6	Bread, Milk, Beer
T7	Milk, Diapers, Cola
T8	Bread, Milk, Diapers, Beer, Eggs

Table 1. Sample transaction database D ($n = 8$ transactions, 6 distinct items).

Apply Apriori to above table with a minimum support threshold of 3 transactions ($\text{min_sup} = 3/8 = 0.375$, i.e. 37.5%).

Step 1 — Candidate 1-itemsets (C_1) and support count

Item	Count	Support	$\geq \text{min_sup}$ (3)?
Bread	6	0.75	Yes
Milk	7	0.875	Yes
Diapers	6	0.75	Yes
Beer	5	0.625	Yes
Cola	3	0.375	Yes
Eggs	2	0.25	No — pruned

Step 2 — Candidate 2-itemsets (C_2) generated by joining L_1 with itself

Because Eggs was pruned in Step 1, no candidate pair in C_2 needs to contain Eggs — this is the anti-monotone pruning already saving work. The remaining 5 items produce $C(5,2) = 10$ candidate pairs, which are then counted against the database:

Candidate Pair	Count	Support	$\geq \text{min_sup}$?
{Bread, Milk}	5	0.625	Yes
{Bread, Diapers}	4	0.50	Yes

Candidate Pair	Count	Support	$\geq \text{min_sup?}$
{Bread, Beer}	4	0.50	Yes
{Bread, Cola}	1	0.125	No
{Milk, Diapers}	5	0.625	Yes
{Milk, Beer}	4	0.50	Yes
{Milk, Cola}	3	0.375	Yes
{Diapers, Beer}	4	0.50	Yes
{Diapers, Cola}	3	0.375	Yes
{Beer, Cola}	1	0.125	No

Table 5. C_2 scan. $L_2 = \{\text{Bread,Milk}\}, \{\text{Bread,Diapers}\}, \{\text{Bread,Beer}\}, \{\text{Milk,Diapers}\}, \{\text{Milk,Beer}\}, \{\text{Milk,Cola}\}, \{\text{Diapers,Beer}\}, \{\text{Diapers,Cola}\}$ — 8 frequent pairs survive; only {Bread,Cola} and {Beer,Cola} are pruned.

Step 3 — Candidate 3-itemsets (C_3) via join + prune

Items are first sorted alphabetically (Beer, Bread, Cola, Diapers, Milk). Candidates are formed by joining two frequent pairs from L_2 that share the same first item, then any candidate whose 2-item subsets are not all present in L_2 is discarded before the database is even touched. Grouping L_2 by leading item gives: Beer $\rightarrow$ {Bread, Diapers, Milk}, Bread $\rightarrow$ {Diapers, Milk}, and Cola $\rightarrow$ {Diapers, Milk}, which yields five raw candidates.

Candidate Triple	Count	Support	$\geq \text{min_sup?}$
{Beer, Bread, Diapers}	3	0.375	Yes
{Beer, Bread, Milk}	3	0.375	Yes
{Beer, Diapers, Milk}	3	0.375	Yes
{Bread, Diapers, Milk}	3	0.375	Yes
{Cola, Diapers, Milk}	3	0.375	Yes

Table 6. C_3 scan. All five candidates pass the subset-pruning check (no eliminations at this stage) and all five meet min_sup , so L_3 contains all five triples — each occurring in exactly 3 of the 8 transactions.

Step 4 — Candidate 4-itemsets (C_4): where pruning and counting diverge

Joining L_3 pairs that share a common 2-item prefix produces exactly one candidate: {Beer, Bread, Diapers, Milk}, from combining {Beer,Bread,Diapers} with {Beer,Bread,Milk}. Its four 3-item subsets — {Beer,Bread,Diapers}, {Beer,Bread,Milk}, {Beer,Diapers,Milk}, {Bread,Diapers,Milk} — are all present in L_3 , so the anti-monotone check cannot eliminate it in advance; it must be counted against the database.

Candidate 4-itemset	Count	Support	$\geq \text{min_sup?}$
{Beer, Bread, Diapers, Milk}	2	0.25	No

Table 6b. C_4 scan. {Beer, Bread, Diapers, Milk} occurs only in T4 and T8 (support 0.25), below the 0.375 threshold, so L_4 is empty.

This is a useful reminder that the anti-monotone property prunes candidates whose subsets are already known to be infrequent, but it cannot predict the support of a candidate whose subsets are all frequent — that still requires an actual database scan.

With L_4 empty, the algorithm terminates.

The complete set of frequent itemsets is $L_1 \cup L_2 \cup L_3$, totalling $5 + 8 + 5 = 18$ frequent itemsets.

From Frequent Itemsets to Association Rules

Once the frequent itemsets are known, rule generation is comparatively cheap.

For every frequent itemset Z with $|Z| \geq 2$, every non-empty proper subset X of Z is considered as a candidate rule antecedent, with $Y = Z - X$ as the consequent, and the rule $X \rightarrow Y$ is kept only if its confidence meets min_confidence .

Taking the frequent itemset {Bread, Milk, Diapers} (support 0.375, i.e. count 3) as an example, with $\text{min_confidence} = 0.6$.

Recall from Table 5 that $\text{count}(\text{Bread}, \text{Diapers}) = 4$, $\text{count}(\text{Bread}, \text{Milk}) = 5$, $\text{count}(\text{Milk}, \text{Diapers}) = 5$, $\text{count}(\text{Bread}) = 6$, $\text{count}(\text{Milk}) = 7$, and $\text{count}(\text{Diapers}) = 6$:

Candidate Rule	Support(rule)	Confidence	Keep (≥ 0.6)?
Bread, Diapers $\rightarrow$ Milk	0.375	$3/4 = 0.75$	Yes
Bread, Milk $\rightarrow$ Diapers	0.375	$3/5 = 0.60$	Yes
Milk, Diapers $\rightarrow$ Bread	0.375	$3/5 = 0.60$	Yes
Bread $\rightarrow$ Milk, Diapers	0.375	$3/6 = 0.50$	No
Milk $\rightarrow$ Bread, Diapers	0.375	$3/7 = 0.429$	No
Diapers $\rightarrow$ Bread, Milk	0.375	$3/6 = 0.50$	No

Table 7. Rule generation from a single frequent 3-itemset. Note the anti-monotone property applies here too: confidence never increases as the antecedent shrinks (moves items to the consequent), so rules can be pruned top-down as well.

Complexity and Practical Notes

- Time complexity in the worst case is exponential in the number of items, but the anti-monotone pruning makes typical sparse retail datasets tractable.
- Apriori requires multiple full passes over the database — one per value of k — which can be I/O-intensive on very large transaction logs; hash-tree structures are commonly used to speed up candidate counting.
- Choice of min_sup is a trade-off: too high and interesting-but-rare patterns (e.g., niche but highly profitable bundles) are missed; too low and the candidate space explodes and produces noisy, low-value rules.

Practical Implementation in Python

The most common tools for MBA in Python are pandas for data preparation and mlxtend for the Apriori algorithm and association rule generation. The example below reproduces the theoretical calculations from Section 3 in code, using the exact same 8-transaction dataset, so the output can be checked against Tables 4–7.

Preparing the Transaction Data

The mlxtend implementation expects a one-hot encoded DataFrame: one row per transaction, one boolean column per item, marking whether that item was present.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# The same 8 transactions used throughout this document
dataset = [
    ['Bread', 'Milk'],
    ['Bread', 'Diapers', 'Beer', 'Eggs'],
    ['Milk', 'Diapers', 'Beer', 'Cola'],
    ['Bread', 'Milk', 'Diapers', 'Beer'],
    ['Bread', 'Milk', 'Diapers', 'Cola'],
    ['Bread', 'Milk', 'Beer'],
    ['Milk', 'Diapers', 'Cola'],
    ['Bread', 'Milk', 'Diapers', 'Beer', 'Eggs'],
]

encoder = TransactionEncoder()
encoded_array = encoder.fit(dataset).transform(dataset)
df = pd.DataFrame(encoded_array, columns=encoder.columns_)
print(df)
```

Mining Frequent Itemsets

```
# min_support = 0.375 reproduces the min_sup = 3/8 threshold from Section 3.3
frequent_itemsets = apriori(
    df,
    min_support=0.375,
    use_colnames=True
)

frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets = frequent_itemsets.sort_values(
    by=['length', 'support'], ascending=[True, False]
)
print(frequent_itemsets)
```

Expected Output

The DataFrame returned contains 18 rows matching Tables 4–6: 5 frequent 1-itemsets, 8 frequent 2-itemsets, and 5 frequent 3-itemsets, including {Bread, Milk, Diapers} at support 0.375. Eggs and any itemset containing it will correctly be absent, and no 4-itemset meets the threshold.

Generating Association Rules

```
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.6
)

rules = rules[['antecedents', 'consequents',
               'support', 'confidence', 'lift']]
rules = rules.sort_values(by='lift', ascending=False)
print(rules.to_string(index=False))
```

Running this on the sample data reproduces the three surviving rules from Table 7 — (Bread, Diapers → Milk), (Bread, Milk → Diapers), and (Milk, Diapers → Bread) — with support 0.375 and confidence 0.75, 0.60 and 0.60 respectively, exactly matching the hand calculation.

Filtering for High-Value Rules

In practice, raw rule lists from real transaction logs can run into the thousands. A standard next step is to filter on lift as well as confidence, since lift screens out rules that only look strong because the consequent item is generally popular (as demonstrated with Bread → Milk in Section 2.3).

```
strong_rules = rules[
    (rules['lift'] > 1.0) &
    (rules['confidence'] >= 0.6)
].sort_values(by=['lift', 'confidence'], ascending=False)

print(strong_rules.to_string(index=False))
```

Visualizing Rules (Optional)

```
import matplotlib.pyplot as plt

plt.scatter(rules['support'], rules['confidence'],
            s=rules['lift'] * 100, alpha=0.6,
            c=rules['lift'], cmap='viridis')
plt.xlabel('Support')
plt.ylabel('Confidence')
plt.title('Association Rules: Support vs Confidence (bubble size/colour = Lift)')
plt.colorbar(label='Lift')
plt.show()
```

This scatter plot is a common way to scan many rules at once: the most actionable rules sit in the top-right (high support and high confidence) with large, brightly coloured bubbles (high lift).

Advantages and Limitations of Apriori

Advantages

- Conceptually simple and easy to explain to non-technical stakeholders.
- The anti-monotone property gives a mathematically guaranteed, exhaustive pruning strategy — no frequent itemset is ever missed.
- Produces human-readable, directly actionable rules (unlike many black-box models).
- Well supported by mature libraries (mlxtend, R's arules, Spark MLlib's FP-Growth/Apriori).

Limitations

- Repeated full database scans (one per itemset size k) are expensive on very large transaction logs.
- Candidate generation can still explode combinatorially when `min_support` is set low or the item catalogue is large.
- Confidence alone can be misleading for rules involving very popular items — lift (or other measures such as conviction) must be checked as well.
- Assumes all items are equally important; does not natively account for price, margin, or quantity (variants such as weighted or quantitative association rule mining address this).
- Produces many rules that are statistically valid but not actionable, so business judgement is still required to filter results.